Labo 17 — Antwoorden met toelichting

Elk antwoord volgt hetzelfde schema: het antwoord, het mechanisme, de nuance of valkuil, een voorbeeld dat je in de terminal kunt nagaan.

Vraag 1 — Chart, release, revision

Antwoord. De **chart** is het pakket: templates, standaardwaarden en metadata, in een map of een `.tgz`. Een **release** is een installatie van die chart onder een naam, in een namespace (`stack` in `lab17`): je kunt dezelfde chart meerdere keren installeren onder verschillende namen. Een **revision** is een genummerde versie van de release: elke `install`, `upgrade` of `rollback` maakt er een. Helm bewaart elke revision (chart + waarden + gerenderd manifest) in een Secret `sh.helm.release.v1.stack.v<n>` van de namespace. Omdat alles in de API server zit, vindt een client alleen, met een kubeconfig, de geschiedenis terug en kan hij een revision opnieuw toepassen: geen Tiller en geen database nodig.

Waarom. Helm 3 verving de server Tiller door deze opslag in de cluster, met de RBAC-rechten van de gebruiker.

Nuance. Standaard bewaart Helm 10 revisions (`--history-max`). Die Secrets met de hand verwijderen vernietigt de geschiedenis; `helm uninstall` verwijdert ze ook (behalve met `--keep-history`).

Voorbeeld.

```
kubectl get secrets -n lab17 -l owner=helm      # sh.helm.release.v1.stack.v1, .v2, …
helm history stack -n lab17
```

Vraag 2 — `--set` en vergeten waarden

Antwoord. `helm get values` toont de **door de gebruiker opgegeven waarden** van de laatste revision: alleen `api.message: v2`. Na het derde commando draait de API met **2 replica's** (de waarde uit `values.yaml`), niet 3: elke `helm upgrade` vertrekt opnieuw van de standaardwaarden van de chart plus wat je **bij dat commando** meegeeft. Om te cumuleren: `helm upgrade stack ./stack --reuse-values --set api.message=v2`, of beide `--set`'s opnieuw geven, of — beter — alles in een `values-<env>.yaml` zetten dat je telkens met `-f` meegeeft.

Waarom. `--set`'s zijn op zichzelf niet persistent; ze worden alleen "onthouden" in de revision, en een nieuwe upgrade erft ze alleen met `--reuse-values`.

Nuance. `--reuse-values` is op zijn beurt verraderlijk: het negeert de nieuwe standaardwaarden van een bijgewerkte chart. De bedrijfspraktijk: een geversioneerd waardenbestand per omgeving, en nooit `--set` "met de hand" in productie.

Voorbeeld.

```
helm upgrade stack ./stack --set api.replicas=3 ; helm upgrade stack ./stack --set
api.message=v2
kubectl get deployment api -o jsonpath='{.spec.replicas}'    # 2
helm get values stack                                        # api: message: v2
```

Vraag 3 — Helm vindt de cluster niet

Antwoord. `helm` zoekt een kubeconfig op de standaardplaats (`~/.kube/config` of `$KUBECONFIG`); die is er niet, dus probeert het het standaardadres `localhost:8080`, waar niets luistert. Correctie: `export KUBECONFIG=/etc/rancher/k3s/k3s.yaml` (of dat bestand naar `~/.kube/config` kopiëren). De `kubectl` van k3s heeft dat probleem niet omdat hij **in het binaire bestand van k3s zit**, dat zijn eigen kubeconfig kent.

Waarom. Helm is een generieke client, net als Lens, k9s of een apart geïnstalleerde `kubectl`; ze lezen allemaal `KUBECONFIG`.

Nuance. Het bestand is leesbaar dankzij `--write-kubeconfig-mode 644` (labo 11); op een bedrijfscluster krijg je de kubeconfig van het platformteam en bevat hij inloggegevens: commit hem nooit.

Voorbeeld.

```
export KUBECONFIG=/etc/rancher/k3s/k3s.yaml ; helm list -A    # traefik, traefik-crd
```

Vraag 4 — De annotatie `checksum/config`

Antwoord. Bij `helm upgrade --set api.message=anders` wordt het template `configmap.yaml` gerenderd met de nieuwe boodschap; zijn `sha256sum` verandert; de annotatie van het Pod-template verandert dus ook; omdat het Pod-template gewijzigd is, start het Deployment een **rolling update** en lezen de nieuwe Pods de nieuwe ConfigMap. Zonder de annotatie zou de ConfigMap bijgewerkt worden maar zou het Deployment, ongewijzigd, zijn Pods met de oude omgeving behouden (labo 14): je zou een `kubectl rollout restart` nodig hebben. Ongewenst geval: de ConfigMap bevat iets dat zonder functionele reden varieert (een tijdstempel, een commentaar, de volgorde van de sleutels): elke upgrade herdeployt dan voor niets.

Waarom. Het Deployment bewaakt alleen zijn eigen `spec.template`; de annotatie "importeert" er de inhoud van de ConfigMap in als vingerafdruk.

Nuance. Dezelfde techniek geldt voor Secrets (`secret.yaml`). Een alternatief is een aparte controller (Reloader) die ConfigMaps observeert en de Deployments herstart die ze mounten.

Voorbeeld.

```
kubectl get deployment api -o jsonpath='{.spec.template.metadata.annotations.checksum/config}'
; echo
helm upgrade stack ./stack --reuse-values --set api.message=anders
kubectl get deployment api -o jsonpath='{.spec.template.metadata.annotations.checksum/config}'
; echo   # anders
```

Vraag 5 — Rollback en PVC

Antwoord. Na `helm rollback stack 1` telt de geschiedenis **vier** revisions: 1, 2, 3 `superseded` en 4 `deployed`, beschreven als `Rollback to 1`. Helm wist niets: teruggaan is een **nieuwe revision** met de inhoud van revision 1. Na `helm uninstall` toont `kubectl get pvc` nog altijd `data-db-0` `Bound`: Helm verwijdert de objecten die het gemaakt heeft, en de PVC werd gemaakt door het **StatefulSet** via `volumeClaimTemplates`, niet door Helm. Dat is gewild: een release deïnstalleren mag geen database vernietigen. De data verdwijnen met `kubectl delete pvc data-db-0` of met het verwijderen van de namespace.

Waarom. Dezelfde logica als `podman compose down` zonder `-v` en als `kubectl delete statefulset` (labo 15).

Nuance. Een chart kan hooks of de annotatie `helm.sh/resource-policy: keep` declareren om andere objecten te beschermen. En `helm rollback` brengt de inhoud van een database niet terug: het beheert alleen manifesten.

Voorbeeld.

```
helm history stack | tail -1     # 4  …  deployed  …  Rollback to 1
helm uninstall stack ; kubectl get pvc    # data-db-0   Bound
```

Vraag 6 — Helm of Kustomize

Antwoord. (a) PostgreSQL en Prometheus: **Helm**, met de officiële charts (Bitnami, prometheus-community) en een `values.yaml` per omgeving: die software wordt met honderden opties geparametreerd, door anderen onderhouden en als pakketten geversioneerd. (b) Vier eigen microservices in drie omgevingen via Argo CD: **Kustomize**: een base per service, een overlay per omgeving, allemaal leesbaar als zuivere YAML in Git, zonder templates om te debuggen, en Argo CD past het rechtstreeks toe. Ze leven samen omdat elk team beide doet: producten van derden installeren en zijn eigen applicaties deployen; Argo CD kan trouwens een Helm chart renderen, en `helm template | kustomize` laat toe een publieke chart te patchen zonder ze te forken.

Waarom. Helm optimaliseert de **verspreiding** (pakket, versie, repository); Kustomize optimaliseert de **leesbaarheid** van varianten van YAML die je zelf bezit.

Nuance. Een heel eenvoudige eigen chart (zoals `stack/`) en een Kustomize-overlay zijn gelijkwaardig; de twist Helm/Kustomize is vooral een teamkwestie. Het echte criterium: wie onderhoudt de YAML, en hoeveel omgevingen.

Voorbeeld.

```
helm template stack ./stack -f stack/values-prod.yaml | grep -c '^kind:'    # 11 gerenderde objecten
kubectl kustomize kustomize/overlays/prod | grep -c '^kind:'                 # 3
```

Vraag 7 — Label ontbreekt op de Pods

Antwoord. De transformer `labels` van Kustomize voegt, met `includeSelectors: false` en niets anders, het label alleen toe aan de **metadata van de objecten** (Deployment, Service), niet aan het `spec.template` van de Pods. Je hebt `includeTemplates: true` nodig zodat het ook de Pod-templates bereikt. `includeSelectors: true` zou het label **aan de selectors** toevoegen (`spec.selector.matchLabels` van het Deployment, `spec.selector` van de Service): de selector van een Deployment is echter **onveranderlijk**; op een bestaand Deployment zou de `apply` falen (`field is immutable`), en zou je het moeten verwijderen en opnieuw maken.

Waarom. Kustomize onderscheidt drie doelen voor een label: het object, zijn Pod-template, zijn selectors; het oude `commonLabels` raakte alle drie, vandaar dat het afgeschaft is.

Nuance. Een *selector*-label moet stabiel blijven zolang het Deployment leeft; een *informatie*-label (`env`, `team`, `version`) gaat in het object en het template, nooit in de selector.

Voorbeeld.

```
labels:
  - pairs: { env: prod }
    includeSelectors: false
    includeTemplates: true
```

```
kubectl get pods -n lab17-prod -L env     # ENV prod
```

Vraag 8 — Twee digests, één image

Antwoord. De lokale digest (`sha256:e1b2…`) is de vingerafdruk van het **manifest zoals Podman het lokaal bijhoudt**; bij de push **hercomprimeert** Podman de lagen en maakt het een ander manifest voor de registry, met vingerafdruk `sha256:b820…`: dat is wat `--digestfile` schrijft en wat de node na de pull in `imageID` ziet. Beide duiden dezelfde inhoud aan, maar alleen de digest **van de registry** is er een die een cluster kan pullen: die pin je vast (`image: localhost:5000/api@sha256:b820…`). Een tag volstaat niet omdat hij beweeglijk is: een nieuwe push op `api:1.1` verandert geruisloos waar `1.1` naar wijst.

Waarom. Een digest is de hash van een precies manifest (lijst van gecomprimeerde lagen + config); de compressie veranderen verandert het manifest, niet de image eens gedecomprimeerd (zelfde `IMAGE ID`, labo 10).

Nuance. `podman pull` vanuit de registry gevolgd door `podman image inspect` geeft ditmaal `b820…`: de lokale digest wordt die van de registry. `skopeo inspect docker://…` bevraagt de registry zonder iets te pullen.

Voorbeeld.

```
podman push --tls-verify=false --digestfile digest.txt localhost:5000/api:1.1 ; cat digest.txt
kubectl get pods -l app=api -o jsonpath='{.items[0].status.containerStatuses[0].imageID}'    # …@sha256:b820…
```

Vraag 9 — HTTP voor een niet-lokale registry

Antwoord. containerd (net als Podman) tolereert **onversleuteld HTTP alleen voor `localhost` / `127.0.0.1`**; voor elke andere host eist het TLS, vandaar `server gave HTTP response to HTTPS client`. Op k3s declareer je de registry in `/etc/rancher/k3s/registries.yaml`:

```
mirrors:
  "registry.intern:5000":
    endpoint: ["http://registry.intern:5000"]
```

en daarna `sudo systemctl restart k3s`: k3s genereert de containerd-configuratie opnieuw bij het opstarten. Op een beheerde cluster raak je de nodes niet aan: de registry **moet** TLS hebben (erkend certificaat), en de inloggegevens gaan in een Secret `kubernetes.io/dockerconfigjson` waarnaar `imagePullSecrets` in de Pod (of de ServiceAccount) verwijst.

Waarom. De pull gebeurt door de kubelet/containerd van de node, met de configuratie van de node; de API server doet er niet aan mee.

Nuance. Hetzelfde bestand dient voor mirrors (`"docker.io": endpoint: [...]`) en voor authenticatie (`configs: … auth: username/password`). Een HTTP-registry, zelfs intern, is een

uitzondering voor het labo: in een bedrijf overal TLS.

Voorbeeld.

```
kubectl create secret docker-registry reg --docker-server=registry.intern --docker-username=ci
--docker-password=…
kubectl patch serviceaccount default -p '{"imagePullSecrets":[{"name":"reg"}]}'
```

Vraag 10 — `helm upgrade --install --wait`

Antwoord. `--install` maakt het commando **idempotent**: eerste uitvoering = `install`, volgende = `upgrade`, dezelfde regel in alle gevallen. `--wait` wacht tot alle Pods `Ready` zijn (probes van labo 12), de Services endpoints hebben, de PVC's gebonden zijn; `--timeout 5m` begrenst dat wachten, waarna de release `failed` wordt en de job faalt: de CI **weet** dat de deployment niet geconvergeerd is. De runner zou een kubeconfig moeten hebben gekoppeld aan een **ServiceAccount** met een RoleBinding `edit` op alleen de doelnamespace (labo 16): nooit de `cluster-admin`-kubeconfig van k3s. Bij mislukking: `helm rollback api` (of `--atomic`, dat het zelf doet), en daarna alarmeren.

Waarom. Zonder `--wait` geeft `helm upgrade` de hand terug zodra de API server de objecten aanvaard heeft, lang voordat de Pods draaien.

Nuance. `--wait` detecteert alleen wat de probes onthullen: een eerlijke readiness (labo 12) is de voorwaarde voor zijn waarde. En een Secret voor de kubeconfig van de runner is een secret als een ander: rotatie, minimale reikwijdte.

Voorbeeld.

```
helm upgrade --install stack ./stack -n lab17 --create-namespace --wait --timeout 2m --atomic
helm history stack      # failed → automatische rollback met --atomic
```

Vraag 11 — GitOps

Antwoord. Bij CI "push" voert een job `kubectl` / `helm` uit naar de cluster: je moet hem inloggegevens toevertrouwen, en de werkelijke toestand van de cluster wijkt uiteindelijk af van wat in Git staat (een `kubectl edit` in nood, een handmatige `helm rollback`). Bij GitOps "pull" leest een agent **in** de cluster (Argo CD, Flux) een Git-repository en past hij voortdurend toe wat die beschrijft; de CI doet alleen nog bouwen, de image pushen en **de tag in Git wijzigen**. Voordelen: elke wijziging is een commit (audit, review via *merge request*), de rollback is een `git revert`, de **afwijking** wordt gedetecteerd en gecorrigeerd (of gemeld). Gevolg: de CI heeft geen enkele toegang meer tot de cluster. Het is de lus van labo 11 — gewenste toestand versus werkelijke toestand — toegepast op de hele Git-repository.

Waarom. Het declaratieve model maakt een externe bron van waarheid mogelijk; de Kubernetes-controller reconcilieert objecten, de GitOps-agent reconcilieert repositories.

Nuance. Secrets gaan niet onversleuteld in Git: Sealed Secrets, SOPS of External Secrets (labo 14). En een "nood"-wijziging met de hand wordt door de agent overschreven: dat is de bedoeling.

Voorbeeld.

```
git commit -am "api: 1.1 -> 1.2 in prod" && git push      # dat is de hele deployment
kubectl get applications -n argocd                        # Synced / OutOfSync (Argo CD)
```

Vraag 12 — Naar een bedrijfscluster

Antwoord. Op EKS/AKS/GKE: `Service: LoadBalancer` krijgt een cloud load balancer met een publiek IP (en een kostprijs) in plaats van ServiceLB; `local-path` bestaat niet — de standaard-StorageClass is een netwerkschijf (`gp3`, `managed-csi`, `pd-balanced`), waardoor de Pod van node kan veranderen; de Ingress controller moet geïnstalleerd of vervangen worden (nginx, ALB, GKE Ingress); de identiteit loopt via IAM/Entra/Google en RBAC gezet door het platform. Op OpenShift komen daar de **Routes** bij (in plaats van of naast de Ingress), de **SCC** die standaard een root-container weigeren (je `USER` van labo 05 wordt verplicht, en de UID wordt opgelegd) en `oc`. Wat niet verandert: Pods, Deployments, Services, ConfigMaps/Secrets, PVC's, probes, RBAC, de chart zelf. Om overdraagbaar te blijven: geen StorageClass hardcoderen (de standaard laten, of een waarde), de Ingress en zijn annotaties parametreerbaar maken, nooit een nodepoort veronderstellen, niet-root draaien, en `resources` declareren.

Waarom. De API is standaard; de integraties (netwerk, opslag, identiteit) worden door elk platform geleverd achter dezelfde objecten.

Nuance. `hostPath`, `hostPort`, `NodePort` en alles wat "één node" veronderstelt, breekt het eerst. Test de chart met `helm template -f values-cloud.yaml` voordat je de cluster hebt.

Voorbeeld.

```
# values.yaml — overdraagbaar
db: { storageClass: "" }        # "" = standaard-StorageClass van de cluster
web: { ingress: { enabled: true, className: "", annotations: {} } }
```

Vraag 13 — `latest` + `rollout restart`

Antwoord. Reproduceerbaarheid: `latest` duidt bij elke push een andere image aan; twee Pods van hetzelfde Deployment kunnen twee versies draaien, en niemand weet welke. Rollback: `rollout undo` brengt een identiek template terug (`latest`), dus **dezelfde kapotte image**; zonder tag is er niets om naar terug te keren. Audit: geen commit, geen Helm-revision zegt wat er draait; `imageID` is de enige getuige. Beveiliging: `Always` + `latest` betekent dat wie ook maar op die tag kan pushen, in productie deployt zonder langs de CI te gaan. Juiste versie: de CI bouwt `api:<git-sha>` (onveranderlijk), pusht, en daarna `helm upgrade --install … --set api.image.tag=<sha> --wait --atomic` (of wijzigt de tag in Git voor Argo CD); in productie de digest; `imagePullPolicy: IfNotPresent` volstaat dan, want de tag beweegt nooit.

Waarom. Een deployment moet een functie zijn van haar invoer (chart + waarden + digest); `latest` haalt de belangrijkste invoer weg.

Nuance. `latest` blijft aanvaardbaar… voor een lokale test op k3s, nooit daarbuiten. En `rollout restart` is nuttig — om een configuratie te herladen, niet om een versie op te leveren.

Voorbeeld.

```
helm upgrade --install stack ./stack --set api.image.tag=$(git rev-parse --short HEAD) --wait --atomic
helm history stack      # elke revision noemt een precieze image
```